

Pilgrim CLI Quick Guide

A short reference for contributors working with scenario files, legal actions, and one-step state transitions.

1. Mental model

- A scenario file is a starting game state.
- `legal-actions` prints the menu of legal moves from that state.
- `--action-index X` refers to the numbered menu item printed by `legal-actions`. It is not a Duty tile number.
- `apply` simulates choosing one legal action once and prints the resulting state.
- `--verbose` adds detailed events, player state, resources, workforce, market, merchant, and invariant output.

Example action menu

If `legal-actions` prints:

```
1. Turn: sow north -> north_east | selected duty: north_east (clerical) | action: clerical_devotion
2. Turn: sow north -> north_east | selected duty: north_east (clerical) | action: clerical_silversmith
3. Turn: sow north -> north_east | selected duty: north_east (clerical) | action: tithe
4. Turn: sow north -> north_east | use building: pulpit to move 1 serf village -> abbey for free |
  selected duty: north_east (clerical) | action: clerical_devotion
5. Turn: sow north -> north_east | use building: pulpit to move 1 serf village -> abbey for free |
  selected duty: north_east (clerical) | action: clerical_silversmith
6. Turn: sow north -> north_east | use building: pulpit to move 1 serf village -> abbey for free |
  selected duty: north_east (clerical) | action: tithe
```

then `--action-index 6` means choose action number 6 from that menu.

2. Core commands

Command	What it does	Typical use
<code>validate</code>	Loads the scenario and checks that it is structurally valid.	Use after creating or editing a scenario file.
<code>legal-actions</code>	Lists all legal actions for the current active player from the scenario state.	Use to find the numbered action index.
<code>apply</code>	Applies one chosen legal action and prints the result. It normally does not overwrite the scenario file.	Use to inspect events and resulting state for one action.
<code>generate-setup</code>	Creates a generated setup scenario with selected market, pilgrimage timeline, and setup metadata.	Use to create deterministic random setups from a seed.
<code>solve</code>	Runs the optimizer/search for a scenario, if supported by the current repo state.	Use when you want the engine to search rather than manually inspect one action.

3. Typical workflow

A simple inspection loop usually looks like this:

```
python3 -m pilgrim.cli validate scenarios/pulpit_active_move_serf_001.json
python3 -m pilgrim.cli legal-actions scenarios/pulpit_active_move_serf_001.json
python3 -m pilgrim.cli apply scenarios/pulpit_active_move_serf_001.json --action-index 6 --verbose
```

Interpretation:

- validate answers: can the scenario be loaded correctly?
- legal-actions answers: what can the active player legally do now?
- apply answers: what happens if the active player chooses this one action?

4. Reading a scenario file from the terminal

To print the raw JSON:

```
cat scenarios/pulpit_active_move_serf_001.json
```

To pretty-print JSON:

```
python3 -m json.tool scenarios/pulpit_active_move_serf_001.json
```

To pretty-print and scroll:

```
python3 -m json.tool scenarios/pulpit_active_move_serf_001.json | less
```

Inside less:

- Press space for the next page.
- Press b for the previous page.
- Type /word to search for a word.
- Press q to quit.

5. Example: Pulpit scenario

For a scenario like `scenarios/pulpit_active_move_serf_001.json`, the scenario sets up player one with:

- an active pulpit building,
- 2 serfs in Village,
- 0 acolytes in Abbey,
- 1 acolyte on the north mancala position,

Running `legal-actions` shows base Clerical actions plus optional Pulpit variants:

```
python3 -m pilgrim.cli legal-actions scenarios/pulpit_active_move_serf_001.json
```

A typical list contains three base Clerical actions and three Pulpit variants:

```
1. ... action: clerical_devotion
2. ... action: clerical_silversmith
3. ... action: tithe
4. ... use building: pulpit ... action: clerical_devotion
5. ... use building: pulpit ... action: clerical_silversmith
6. ... use building: pulpit ... action: tithe
```

Applying action 6:

```
python3 -m pilgrim.cli apply scenarios/pulpit_active_move_serf_001.json --action-index 6 --verbose
```

Expected highlights:

```
BUILDING_BONUS: pulpit moved 1 serf village -> abbey for free
WORKFORCE_MOVE: player_one moved 1 serf village -> abbey for free
SOWING: picked up 1 from north; route north -> north_east
DUTY_RESOLUTION: selected north_east (clerical); mode tithe
```

Expected state change:

- Village decreases from 2 to 1.
- Abbey increases from 0 to 1.
- Wheat remains unchanged because Pulpit is free.

6. Finding an action index automatically

For repeated inspection, you can ask the shell to find the first action containing a phrase.

```
SCENARIO=scenarios/pulpit_active_move_serf_001.json
IDX=$(python3 -m pilgrim.cli legal-actions "$SCENARIO" | awk '/use building: pulpit/ {sub(/\./,"", $1); print $1; exit}')
echo "Using action index: $IDX"
python3 -m pilgrim.cli apply "$SCENARIO" --action-index "$IDX" --verbose
```

This is useful when action ordering changes after new rules are added, but the action summary still contains a stable phrase.

7. Validating blocked cases

For scenarios intended to prove a building is unavailable, search the legal-action list for that building name.

```
for SCENARIO in scenarios/pulpit_merchant_none_no_hire_001.json scenarios/pulpit_insufficient_hire_resource_001.json scenarios/pulpit_donated_no_modifier_001.json scenarios/pulpit_not_live_no_modifier_001.json scenarios/pulpit_no_village_serf_no_modifier_001.json
do
    echo "
--- $SCENARIO ---"
    python3 -m pilgrim.cli legal-actions "$SCENARIO" | grep -i "pulpit" || echo "OK: no Pulpit variants"
done
```

8. Generated setup example

To generate a deterministic setup from a seed:

```
python3 -m pilgrim.cli generate-setup --players 2 --seed 1 --output /tmp/pilgrim_setup_seed1.json
```

Then validate and inspect it:

```
python3 -m pilgrim.cli validate /tmp/pilgrim_setup_seed1.json
python3 -m pilgrim.cli legal-actions /tmp/pilgrim_setup_seed1.json
```

Generated setup files include the selected building market, building live rounds, pilgrimage timeline metadata, and setup-sow state.

9. Quick reference

Need	Command
Check scenario validity	<code>python3 -m pilgrim.cli validate scenarios/example.json</code>
List legal actions	<code>python3 -m pilgrim.cli legal-actions scenarios/example.json</code>
Apply action 6	<code>python3 -m pilgrim.cli apply scenarios/example.json --action-index 6 --verbose</code>
Pretty-print scenario JSON	<code>python3 -m json.tool scenarios/example.json</code>
Generate setup	<code>python3 -m pilgrim.cli generate-setup --players 2 --seed 1 --output /tmp/setup.json</code>

Draft v0.1 - intended to be extended as the Pilgrim CLI and scenario workflow evolve.